Redundancy And Scaling Audit

Current storage duplication, dead features, and high-impact simplifications

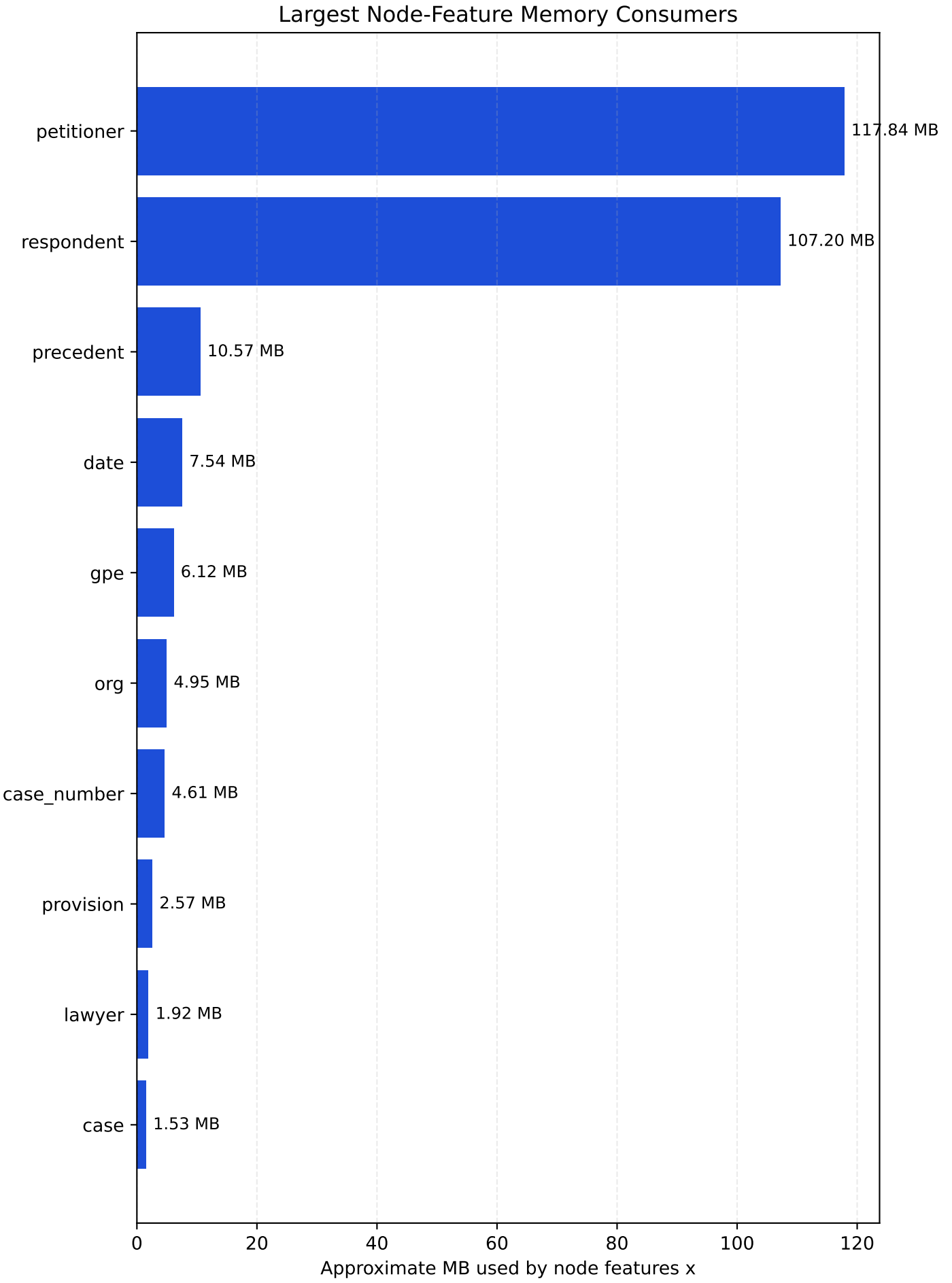
- Total node-feature storage in the live graph is about 273.69 MB; edge_index tensors add about 10.99 MB

- Petitioner + respondent nodes alone account for about 225.04 MB of node features

- This audit distinguishes exact duplicates from intentional summarization and identifies which changes matter most for scaling

Redundancy And Scaling Audit

Current live graph snapshot from data/graph_cache/case_star_global_graph.pt



```
Redundancy And Scaling Audit
============================

This audit separates exact duplication from acceptable summarization. Some repeated information is
intentional because it shortens message-passing paths; other parts are exact duplicates or currently-
dead feature slots that can be removed with no loss of information.

- Current node-feature tensor footprint: about 273.69 MB
- Current directed edge_index footprint: about 10.99 MB
- Party nodes alone (petitioner + respondent) use about 225.04 MB of node-feature storage, which is
  about 82.2% of all x memory
- Exact duplicate entity scalar slot cost now: about 0.678 MB
- Dead text scalar tail cost now: about 0.077 MB
- Case-level combined text embedding duplication cost now: about 1.479 MB

1. High-Impact Scaling Risk: Case-Local Party Nodes
---------------------------------------------------

- Live counts: petitioner=78007 and respondent=70965.
- These nodes are not exact duplicates in the strict sense, but they are by far the main storage hotspot
  because they remain case-local and therefore do not merge across cases.
- Average per case in the live cache: petitioner=77.23, respondent=70.26.
- If your next scaling step is mostly about storage and training throughput, party handling is the first
  place to simplify.

2. Exact Duplicate Feature: mention_count vs local_case_frequency
-----------------------------------------------------------------

- Entity slot 384 stores mention_count / 100.
- Entity slot 390 stores local_case_frequency / 100.
- In the current implementation both are populated from metadata['mention_count'], so they are
  numerically identical for every entity node.
- This is a true no-information-gain duplicate. Removing one of the two saves one float per entity node
  with no semantic loss.

3. Dead Feature Tail On Text Nodes
----------------------------------

- Text-node slots 388 through 395 are allocated for citation and bridge counts.
- The builder initializes cited_statute_count, cited_provision_count, cited_precedent_count,
  petitioner_lawyer_count, defence_lawyer_count, petitioner_count, respondent_count, and judge_count to
  0 on section nodes.
- No later step populates those fields, so the current cached graph has those slots equal to 0 for every
  preamble, facts, and arguments node.
- This is dead capacity rather than harmful duplication. It does not change model behavior, but it does
  make the feature tail wider than necessary.

4. Semantic Duplication: Case Text Embedding And Section Text Embeddings
------------------------------------------------------------------------

- Every case node stores one embedding of the concatenated retained text.
- The same case can also store up to three section embeddings on preamble/facts/arguments nodes.
- This is not an exact tensor duplicate because one vector is a whole-case summary and the others are
  section-level summaries, but it is still repeated representation of the same underlying text.
- Storage impact today is modest compared with party nodes, but if you want a cleaner architecture you
  should decide whether you want both whole-case and section-level text embeddings or only one of those
  abstractions.

5. Small But Real Redundancy: Text Type Flags
---------------------------------------------
```

- Text-node slots 385, 386, and 387 are one-hot flags for preamble/facts/arguments.
- The model already knows the node type because the graph is heterogeneous, the input projection is node-type-specific, and HGTConv is relation-aware.
- These flags are therefore redundant in a conceptual sense. They are cheap, so they are not your main scaling problem, but they can be removed for a cleaner feature design.

6. Not Duplicate, But High-Fanout Context

forward relation	live edge count
case|mentions_gpe|gpe	116649
case|has_case_number|case_number	22816
case|has_date|date	16303
case|mentions_org|org	19565

- These are not duplicates by themselves, but they increase connectivity quickly and can dominate message passing.
- If extraction quality on gpe/date/case_number/org is noisy, these relations can add cost without proportional signal.
- If you need a lighter graph, pruning or capping these relation families is a much higher-value change than removing a few duplicate scalar slots.

7. Recommended Change Order For Scaling

- Priority 1: compress or prune petitioner/respondent nodes. Options include top-k parties per case, one pooled petitioner node plus one pooled respondent node, or temporarily dropping party nodes in the first large-scale build.
- Priority 2: choose between whole-case text embedding and section-level text nodes if you want to reduce semantic duplication. Keeping both is defensible, but it is not minimal.
- Priority 3: remove one of mention_count/local_case_frequency on entity nodes. This is a true duplicate and easy to clean up.
- Priority 4: either populate the text-node count fields from actual edges or delete those scalar slots from the text tail.
- Priority 5: drop the text-node one-hot type flags if you want the feature design to rely purely on heterogeneous node typing.
- Priority 6: review gpe/date/case_number/org relation families for extraction noise and cap them if they are mostly surface metadata rather than task signal.

8. Practical Bottom Line

- The graph will not crash because of the exact duplicate scalar slots. Those are small.
- The real scaling pressure today comes from graph breadth: very many case-local party nodes and high-fanout context relations.
- If you want the cleanest non-redundant design, the first architectural question is not whether to remove one duplicate scalar. It is whether you really need case-local party expansion plus whole-case text embeddings plus section text nodes all at the same time.